When a user logs in, Laravel stores their session using secure cookies. If they attempt to access a protected route, the system verifies their session and grants or denies access accordingly.

13.4 Managing Users and Profiles

Beyond login and registration, real-world applications often require users to manage their profiles. Laravel allows developers to easily implement features for updating user information, changing passwords, and even uploading avatars. This functionality typically involves creating a user controller with update methods and protected routes accessible only to logged-in users.

13.5 Securing Routes with Middleware

Protecting routes is essential to restrict access to specific parts of your application. Laravel uses middleware to intercept requests and determine whether a user should be granted access. The `auth` middleware is commonly used to enforce authentication. You can apply it to individual routes or entire route groups.

```
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index']);
});
```

Unauthenticated users attempting to access these routes will be redirected to the login page.

> ***Hands-on Lab**: Create a `/dashboard` route protected by the `auth` middleware and customize the content for logged-in users.*

13.6 Implementing Authorization with Gates and Policies

Laravel's authorization system is based on **Gates** and **Policies**. Gates are closures that determine if a user is authorized to perform a specific action, whereas Policies are classes that group authorization logic around a particular model.

Policies can be automatically mapped to Eloquent models, allowing you to centralize your access control. For example, a `PostPolicy` can define who can update